

DETECTING DEFECTS IN PRINTED CIRCUITS BOARDS USING YOLO-BASED METHOD

A Mini Project Report

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Engineering

in

Computer Science and Engineering

By

THATIPUDI BAVYA SRI SAI

2456-23-733-413

RAINI NOUMI YADAV

2456-23-733-403

SNEHA SATAPATHY

2456-23-733-410

**Under the guidance of
Dr. Padmalaya Nayak
Professor & Head,
CSE**



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Gokaraju Lailavathi Engineering College

(Affiliated to Osmania University, Approved by AICTE)

Nizampet, Hyderabad-500090

(2025-2026)

Department of Computer Science and Engineering
Gokaraju Lailavathi Engineering College

(Affiliated to Osmania University, Approved by AICTE)

Nizampet, Hyderabad-500090

(2024-2025)



CERTIFICATE

This is to Certify that a Mini Project report entitled “**Detecting Defects in Printed Circuit Boards Using YOLO-Based Method**” is being submitted by Thatipudi Bavya Sri Sai (2456-23-733-413), Raini Noumi Yadav (2456-23-733-403), Sneha Satapathy (2456-23-733-410) in partial fulfillment of the requirement of the award for the degree of Bachelor of Engineering in “Computer Science and Engineering” O.U., Hyderabad during the year 2025-2026 is a record of bonafide work carried out by them under my guidance. The results presented in this project have been verified and are found to be satisfactory.

Project Guide

Dr. Padmalaya Nayak

Professor & Head

Dept. of CSE

H.O.D

Dr. Padmalaya Nayak

Professor & Head,

Dept. of CSE

External Examiner(s)



Department of Computer Science and Engineering
Gokaraju Lailavathi Engineering College

(Affiliated to Osmania University, Approved by AICTE)

Nizampet, Hyderabad-500090

(2024-2025)

DECLARATION

We, Thatipudi Bavya Sri Sai bearing Ht.No.2456-23-733-413, Raini Noumi Yadav bearing Ht.No.2456-23-733-403 and Sneha Satapathy bearing Ht.No.2456-23-733-410 hereby certify that the minor project entitled “**Detecting Defects in Printed Circuit Boards Using YOLO-Based Method**” is submitted in the partial fulfilment of the required for the award of the degree of Bachelor of Engineering in Computer Science and Engineering.

This is a record work carried out by us under the guidance of Dr. Padmalaya Nayak, Professor & Head, CSE, Gokaraju Lailavathi Engineering College, Nizampet. The results embodied in this report have not been reproduced/copied from any source. The results embodied in this report have not been submitted to any other university or institute for the award of any other degree or diploma.

THATIPUDI BAVYA SRI SAI

2456-23-733-413

RAINI NOUMI YADAV

2456-23-733-403

SNEHA SATAPATHY

2456-23-733-410

ACKNOWLEDGEMENT

It is our privilege and pleasure to express my profound sense of respect, gratitude and indebtedness to our guide Dr. Padmalaya Nayak, Professor & Head, Department of Computer Science and Engineering, Gokaraju Lailavathi Engineering College, for her inspiration, guidance, cogent discussion, constructive criticisms and encouragement throughout this dissertation work.

We express our sincere thanks to Project Coordinator N. Swathi, Assistant Professor, Department of Computer Science and Engineering, Gokaraju Lailavathi Engineering College, for her valuable suggestions and constant helping in completing the work.

We express our sincere thanks to Dr. Padmalaya Nayak, Professor and Head of the Department of Computer Science and Engineering, Gokaraju Lailavathi Engineering College, for her valuable guidance, support and encouragement throughout the completion of this work.

We express our sincere thanks to Dr. A. Sai Hanuman, Principal, Gokaraju Lailavathi Engineering College, Nizampet, Hyderabad, for his encouragement and constant help.

We extend our sincere thanks to all the teaching and non-teaching staff of Computer Science and Engineering Department for their support and encouragement.

Last but not least, we wish to acknowledge my friends and family members for giving moral strength moral strength and helping us to complete this dissertation.

ABSTRACT

Printed Circuit Boards, or PCBs, are important parts of most modern electronics. They act as a base that connects various electronic components together. When making PCBs, problems like open circuits, short circuits, too much copper, and mouse bites can happen. These issues can make electronics less reliable and cause them to work poorly. In the past, people checked these boards by hand, which was a slow and hard process that often made mistakes, especially when there were a lot of boards to check. To make this process faster and better, people have started using computer vision and deep learning to automatically find defects. Scientists have used techniques like convolutional neural networks (CNNs) to find problems in PCBs using datasets such as the DeepPCB dataset. Even though these methods have worked well, it is still hard to find small or complicated defects. Sometimes the models do not pick up enough details or focus on the right parts. In this study, we propose a new method for detecting PCB defects using the YOLO object detection system. The YOLO model helps in identifying defect regions accurately by detecting and localizing defects in PCB images. The model is trained and tested using the DeepPCB dataset. This research aims to create an automatic inspection system that can quickly and accurately find defects, which would be helpful for real-world PCB manufacturing and improve detection accuracy compared to existing methods.

CONTENTS

Chapter-1 INTRODUCTION	1
1.1 MOTIVATION... ..	1
1.2 PROBLEM STATEMENT.....	1
1.3 PROJECT OBJECTIVE	1
Chapter-2 LITERATURE SURVEY	2
2.1 DEEPPCB: A DATASET FOR PCB DEFECT DETECTION.....	2
2.2 YOLO-BASED PCB DEFECT DETECTION	2
2.3 CONVOLUTIONAL NEURAL NETWORK-BASED PCB DEFECT DETECTION.....	2
2.4 DEEP LEARNING FOR AUTOMATED VISUAL INSPECTION	3
2.5 CONCLUSION.....	3
Chapter-3 SOFTWARE REQUIREMENT SPECIFICATION	4
3.1 SOFTWARE REQUIREMENTS	4
3.2 HARDWARE REQUIREMENTS.....	4
3.3 FUNCTIONAL REQUIREMENTS	5
3.4 NON-FUNCTIONAL REQUIREMENTS	5
Chapter-4 SYSTEM DESIGN.....	6
4.1 SYSTEM DESIGN	6
4.2 PROPOSED METHODOLOGY	7
4.3 UML DIAGRAMS	8
4.3.1 CLASS DIAGRAM.....	10

4.3.2 USE CASE DIAGRAM	11
4.3.3 COMPONENT DIAGRAM... ..	12
4.3.4 SEQUENCE DIAGRAM... ..	13
4.3.5 ACTIVITY DIAGRAM... ..	14
4.4 TECHNOLOGY DESCRIPTION	15
Chapter-5 IMPLEMENTATION	18
5.1 IMPLEMENTATION	18
5.2 MODULES	21
5.3 EXECUTABLE CODE	23
Chapter-6 RESULTS.....	29
Chapter-7 CONCLUSION.....	32
7.1 CONCLUSION... ..	32
7.2 FUTURE SCOPE... ..	32
REFERENCES.....	34
PLAGIARISM TEST... ..	35

SNO.	LIST OF FIGURES	PAGE NO.
4.2.1	Proposed Methodology	7
4.3.1	UML Hierarchy Diagrams	8
4.3.1.1	Class Diagram	10
4.3.2.1	Use Case Diagram	11
4.3.3.1	Component Diagram	12
4.3.4.1	Sequence Diagram	13
4.3.5.1	Activity Diagram	14
6.1	User Input(Budget, image)	30
6.2	Recommended Output (Image)	30

CHAPTER-1

INTRODUCTION

1.1 MOTIVATION

The rapid growth of the electronics industry has increased the demand for reliable and high-quality Printed Circuit Boards (PCBs) [1], [2]. Even a small manufacturing defect can affect the performance and reliability of an electronic device. Manual inspection requires skilled professionals and consumes significant time, making it difficult for manufacturers to maintain high production rates.

Deep learning-based object detection provides an efficient alternative by automatically identifying defects with high accuracy [3], [4]. Motivated by these advantages, this project aims to develop an intelligent PCB defect detection system using YOLOv8 that can perform real-time inspection while reducing human effort, inspection time, and production cost.

1.2 PROBLEM STATEMENT

PCB manufacturing industries produce thousands of circuit boards every day. Identifying defects manually is challenging because defects are often very small and difficult to notice. Manual inspection may also lead to inconsistent results due to fatigue and human error. Existing traditional image processing techniques are less effective when defects vary in size, shape, and location [3], [4].

Therefore, there is a need for an automated system capable of accurately detecting different PCB defects in real time [4]. This project proposes a YOLO-based defect detection system that automatically identifies multiple PCB defects from images using deep learning techniques.

1.3 PROJECT OBJECTIVE

The main objective of this project is to develop an intelligent PCB defect detection system using the YOLOv8 deep learning model. The proposed system aims to:

Detect PCB defects automatically from input images.

Identify six common defect types: Open, Short, Mousebite, Spur, Copper, and Pin-hole.

Improve inspection accuracy while reducing manual effort.

Provide real-time prediction using a Streamlit web application.

Assist manufacturers in improving PCB quality and production efficiency.

CHAPTER 2

LITERATURE SURVEY

The increasing use of Artificial Intelligence and Deep Learning has significantly improved automated visual inspection in manufacturing industries. Several researchers have proposed different approaches for PCB defect detection using machine learning, image processing, and deep learning models. This chapter reviews some important research works related to PCB defect detection and explains the motivation behind selecting the YOLO-based approach.

2.1 DEEPPCB: A DATASET FOR PCB DEFECT DETECTION

DeepPCB is one of the most widely used public datasets for PCB defect detection research [1]. It contains image pairs consisting of template images and defective PCB images along with annotations for six defect categories: Open, Short, Mousebite, Spur, Copper, and Pin-hole. The dataset provides high-quality annotated images that enable deep learning models to learn defect characteristics effectively [1]. Due to its standard annotations and industrial relevance, DeepPCB has become a benchmark dataset for PCB defect detection research [1].

2.2 YOLO-BASED PCB DEFECT DETECTION

YOLO (You Only Look Once) is a real-time object detection algorithm capable of detecting multiple objects in a single forward pass [4]. Recent versions, particularly YOLOv8, provide high detection accuracy with fast inference speed [4]. Several researchers have successfully applied YOLO models for industrial defect detection because of their ability to localize small defects accurately while maintaining real-time performance. These advantages make YOLOv8 suitable for PCB inspection applications[4].

2.3 CONVOLUTIONAL NEURAL NETWORK (CNN)-BASED PCB DEFECT DETECTION

Earlier PCB defect detection systems mainly relied on Convolutional Neural Networks (CNNs) [3]. CNN-based methods extract image features automatically and classify defect regions effectively [3]. Although CNNs achieve good classification performance, they generally require separate localization techniques and often have slower inference compared to modern object detection models. This limitation encouraged researchers to adopt single-stage detectors like YOLO [4].

2.4 DEEP LEARNING FOR AUTOMATED VISUAL INSPECTION

Deep learning has transformed industrial quality inspection by enabling automatic detection of defects from images [3], [4]. Models such as Faster R-CNN, SSD, RetinaNet, and YOLO have been applied successfully in manufacturing industries[3], [4]. These models reduce inspection time, improve detection accuracy, and minimize human errors. Among these approaches, YOLOv8 provides an excellent balance between speed and accuracy, making it suitable for real-time PCB defect detection [4].

2.5 CONCLUSION

The literature survey indicates that deep learning methods outperform traditional image processing techniques for PCB defect detection. The DeepPCB dataset provides a reliable benchmark for training and evaluation, while YOLOv8 offers high accuracy and real-time performance. Based on these findings, the proposed system adopts the YOLOv8 model for automatic detection of PCB defects.

CHAPTER-3

SOFTWARE REQUIREMENT SPECIFICATION

This chapter gives an overview of the software and hardware components required for the development and deployment of the proposed PCB defect detection system.

3.1 SOFTWARE REQUIREMENTS

- Windows 11 OS or Ubuntu 20.04+
- Python 3.10 or above
- Visual Studio Code
- Ultralytics YOLOv8
- PyTorch
- Streamlit
- OpenCV
- NumPy
- Pandas
- Pillow (PIL)

3.2 HARDWARE REQUIREMENTS

- Desktop/Laptop (Intel Core i5/Ryzen 5 processor or above)
- Minimum 8 GB RAM (16 GB Recommended)
- SSD Storage (Minimum 20 GB Free Space)
- NVIDIA GPU or Kaggle T4 GPU (for model training)
- Internet Connection

3.3 FUNCTIONAL REQUIREMENTS

These are the requirements that describe the basic functionalities provided by the proposed PCB defect detection system. The system allows users to upload PCB images and automatically detects defects using a trained YOLOv8 model. The detected defects are highlighted using bounding boxes, and a summary of detected defect types is displayed to the user.

The functional requirements include:

- PCB Image Upload Module
- PCB Defect Detection
- Defect Classification
- Display of Original PCB Image
- Display of Detected PCB Image
- Defect Summary Generation
- Defect Distribution Bar Chart
- Download Detection Result

3.4 NON-FUNCTIONAL REQUIREMENTS

These are the quality attributes that the system should satisfy to ensure efficient performance and usability. They mainly focus on the reliability and overall quality of the proposed application.

They include:

- Accuracy
- Reliability
- Security
- Maintainability
- Scalability
- Portability
- User Friendliness
- Performance

CHAPTER-4 SYSTEM DESIGN

4.1 System Design

The system design describes the overall architecture and workflow of the proposed PCB defect detection system. The application uses the YOLOv8 object detection model integrated with a Streamlit web application to perform real-time defect detection. The uploaded PCB image is processed by the trained model, and the detected defects are displayed with their corresponding labels and confidence scores.

There are two kinds of design documents developed in this phase:

High-Level Design (HLD)

- Brief description of every module
- Overall workflow of the PCB defect detection system
- Interaction between different modules
- Technology stack used for implementation
- System architecture and data flow

Low-Level Design(LLD)

- Detailed functionality of each module
- Model loading and prediction process
- User Interface implementation
- Detection result generation
- Error handling and validation

4.2 PROPOSED METHODOLOGY:

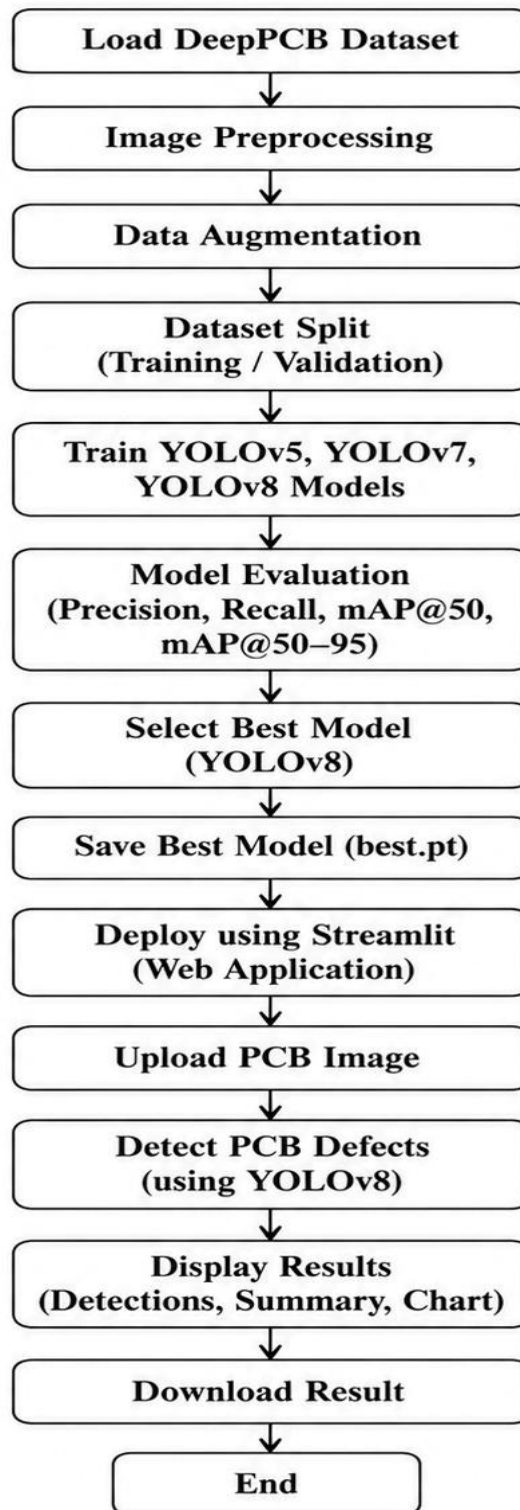


Figure-4.2.1 Proposed Methodology

4.3 UML Design:

Unified Modeling Language (UML) is a general purpose modeling language. The main aim of UML is to define a standard way to visualize the way a system has been designed.

UML is not a programming language; it is rather a visual language. We use UML diagrams to portray the behavior and structure of a system, UML helps software engineers, businessmen and system architects with modeling, design and analysis. The Object Management Group (OMG) adopted Unified Modeling Language as a standard in 1997. It's been managed by OMG ever since. International Organization for Standardization (ISO) published UML as an approved standard in 2005. UML has been revised over the years and is reviewed periodically.

Do we really need UML?

- Complex applications need collaboration and planning from multiple teams and hence require a clear and concise way to communicate amongst them.
- Businessmen do not understand code. So UML becomes essential to communicate with non programmer's essential requirements, functionalities and processes of the system.
- UML is linked with object oriented design and analysis. UML makes the use of elements and forms associations between them to form diagrams. Diagrams in UML can be broadly classified as:

The Primary goals in the design of the UML are as follows:

- Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
- Provide extendibility and specialization mechanisms to extend the core concepts.
- Be independent of particular programming languages and development process.
- Provide a formal basis for understanding the modeling language.
- Encourage the growth of OO tools market.
- Support higher level development concepts such as collaborations, frameworks, patterns and components.
- Integrate best practices.

Types of UML Diagrams:

Structural Diagrams:

Capture static aspects or structure of a system. Structural Diagrams include: Component Diagrams, Object Diagrams, Class Diagrams and Deployment Diagrams.

Behavior Diagrams:

Capture dynamic aspects or behavior of the system. Behavior diagrams include: Use Case Diagrams, State Diagrams, Activity Diagrams and Interaction Diagrams.

The image below shows the hierarchy of diagrams according to UML

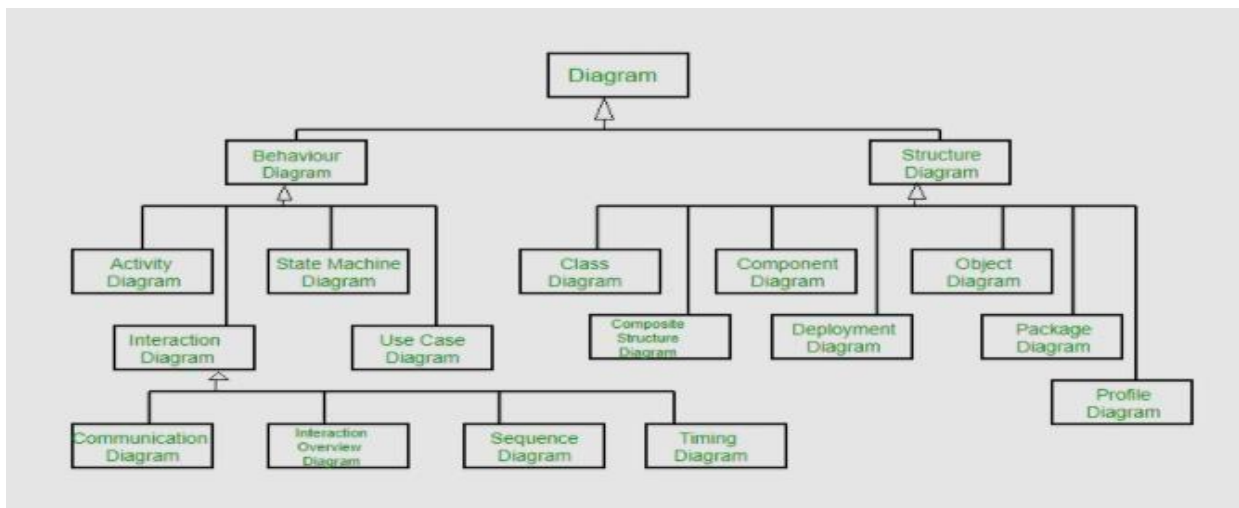


Figure-4.3.1 UML Hierarchy diagrams

4.3.1 CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

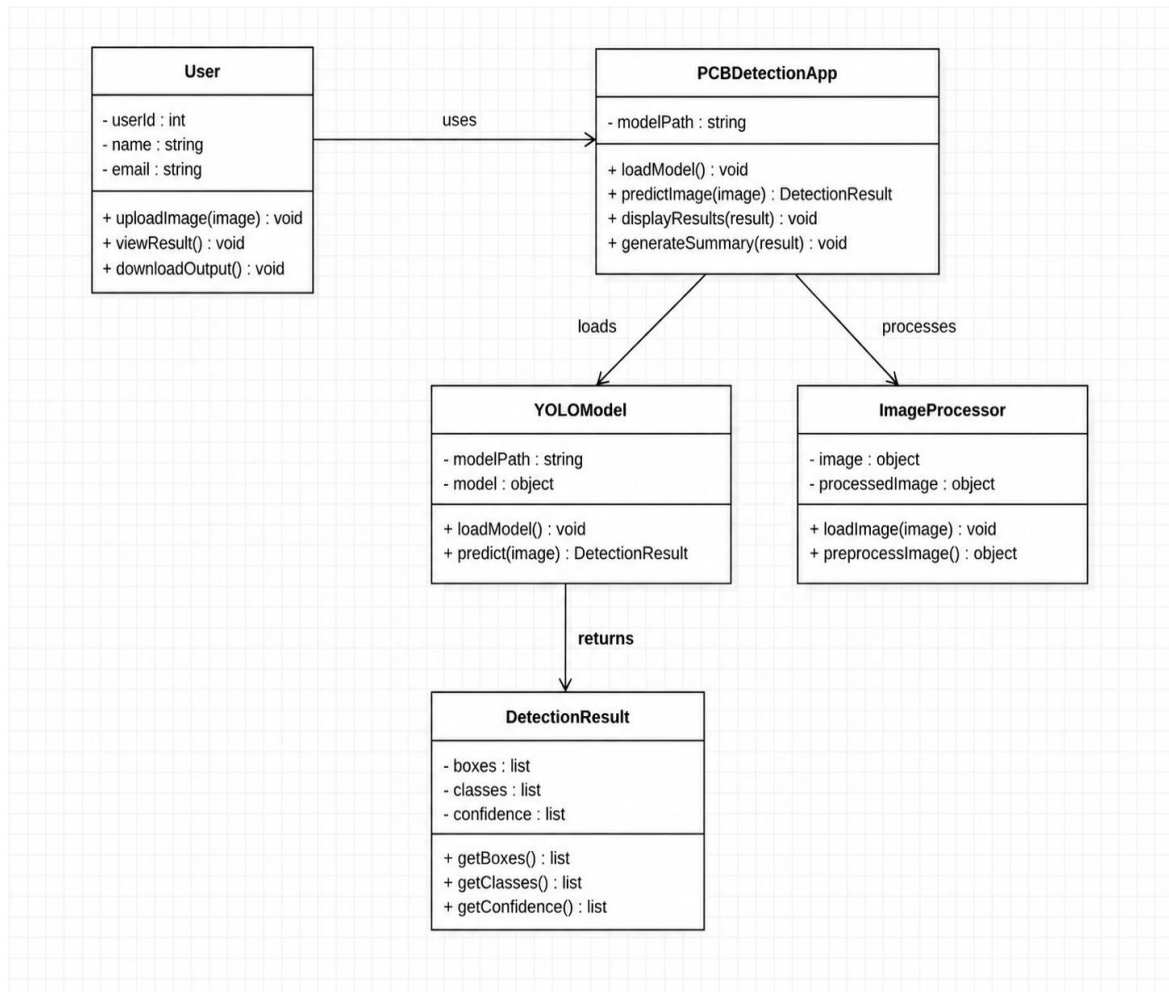


Figure-4.3.1.1 Class Diagram

4.3.2 USE CASE DIAGRAM:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

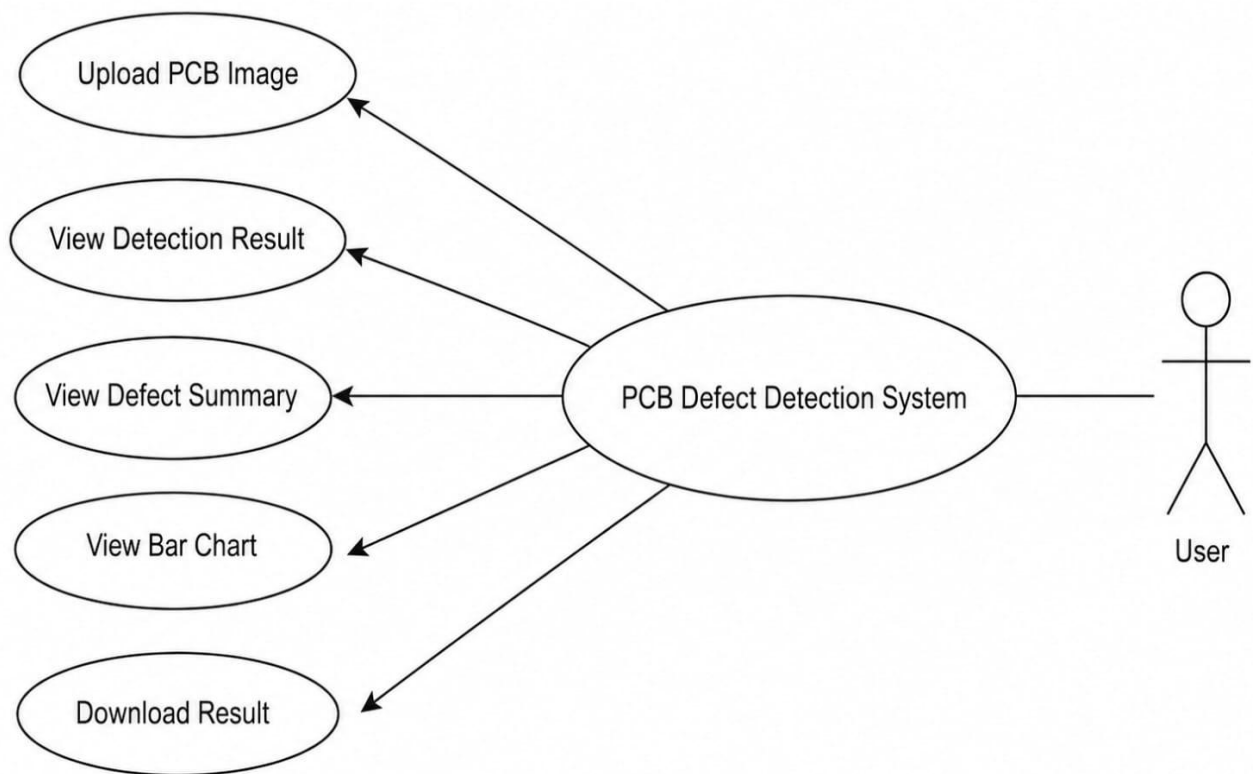


Figure-4.3.2.1 Use Case Diagram

4.3.3 COMPONENT DIAGRAM:

A component diagram, also known as a UML component diagram, describes the organization and wiring of the physical components in a system. Component diagrams are often drawn to help model implementation details and double-check that every aspect of the system's required functions is covered by planned development.

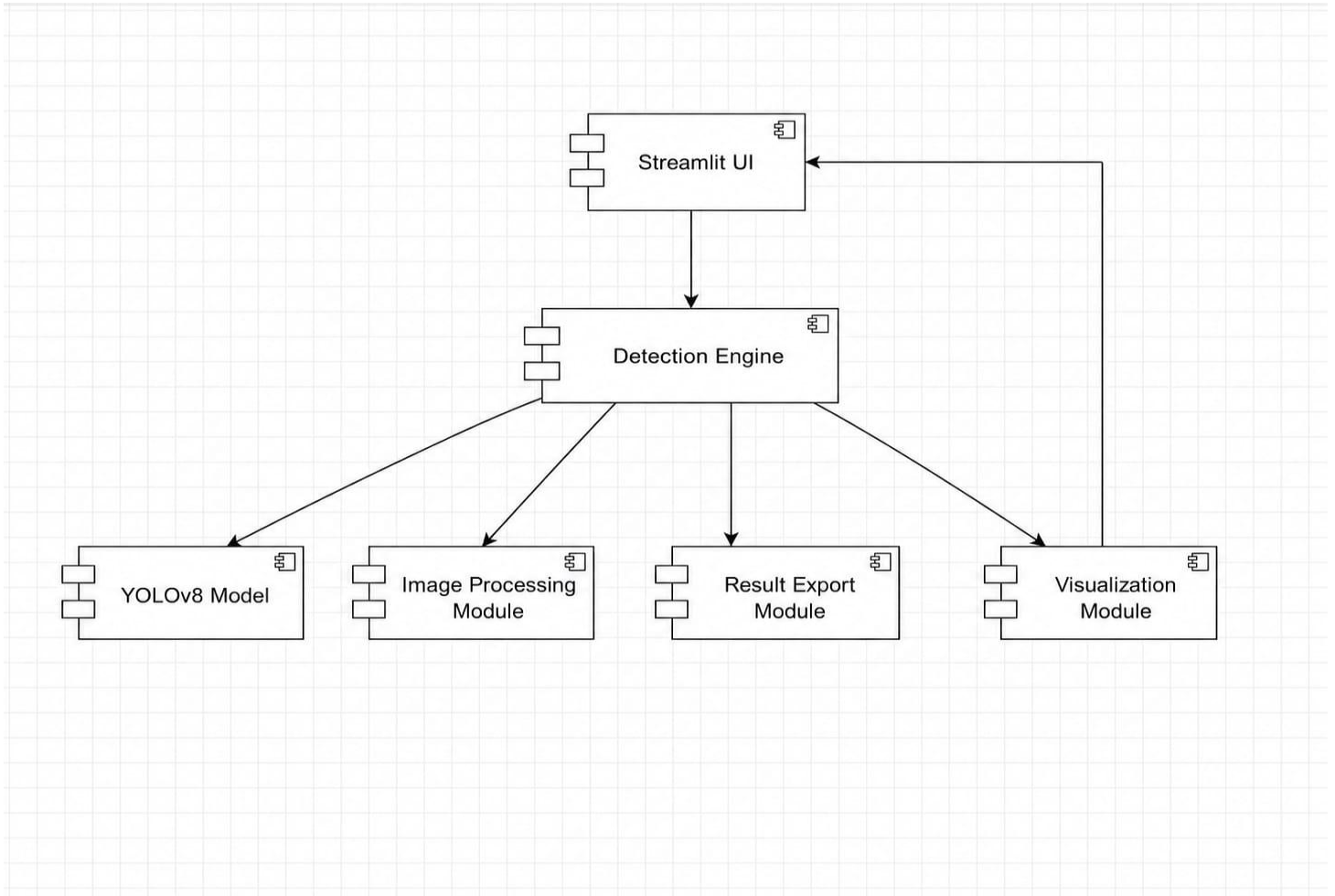


Figure-4.3.3.1 Component Diagram

4.3.4 SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

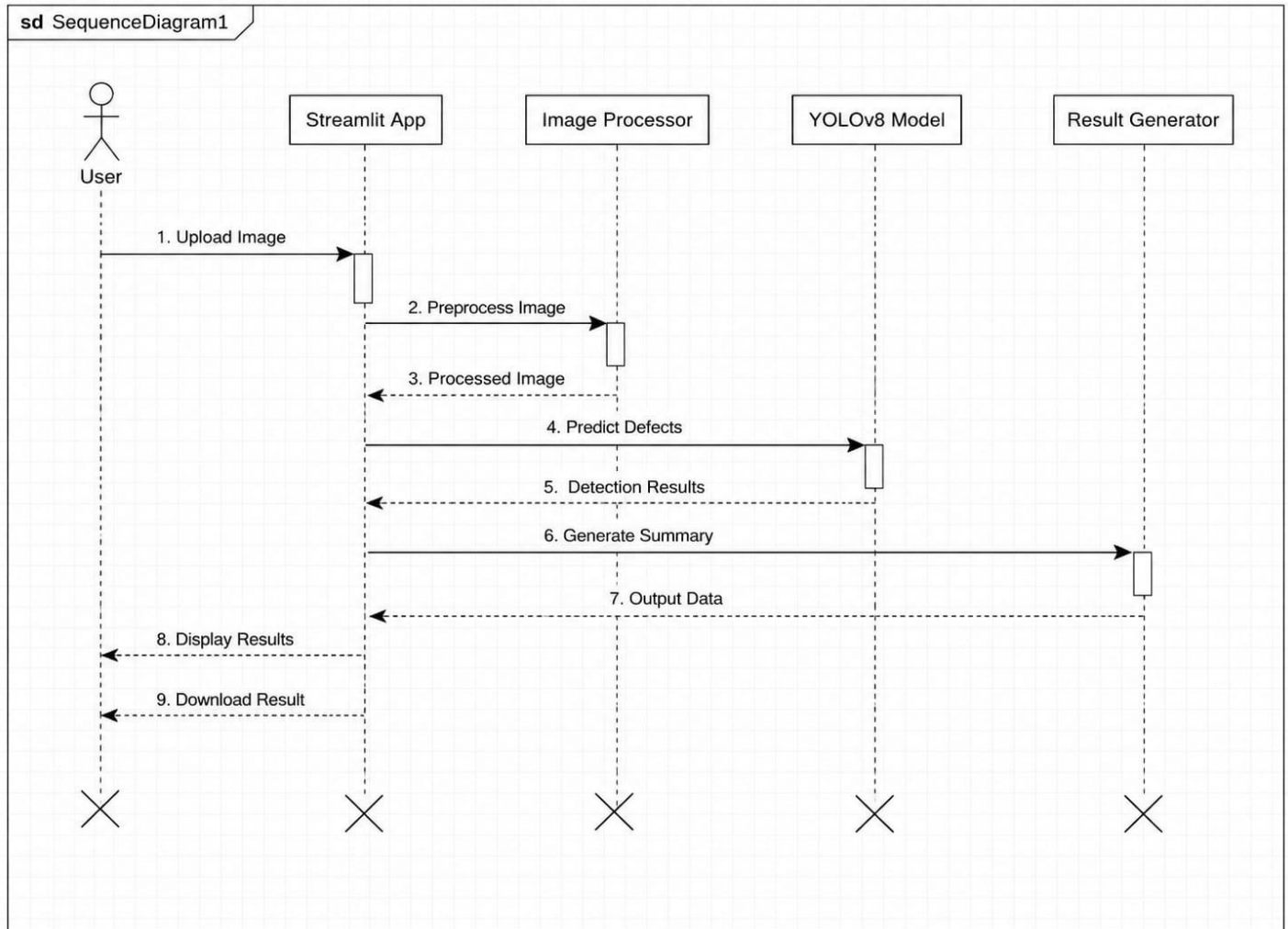


Figure-4.3.4.1 Sequence Diagram

4.3.5 ACTIVITY DIAGRAM:

In UML, an activity diagram is used to display the sequence of activities. Activity diagrams show the workflow from a start point to the finish point detailing the many decision paths that exist in the progression of events contained in the activity.

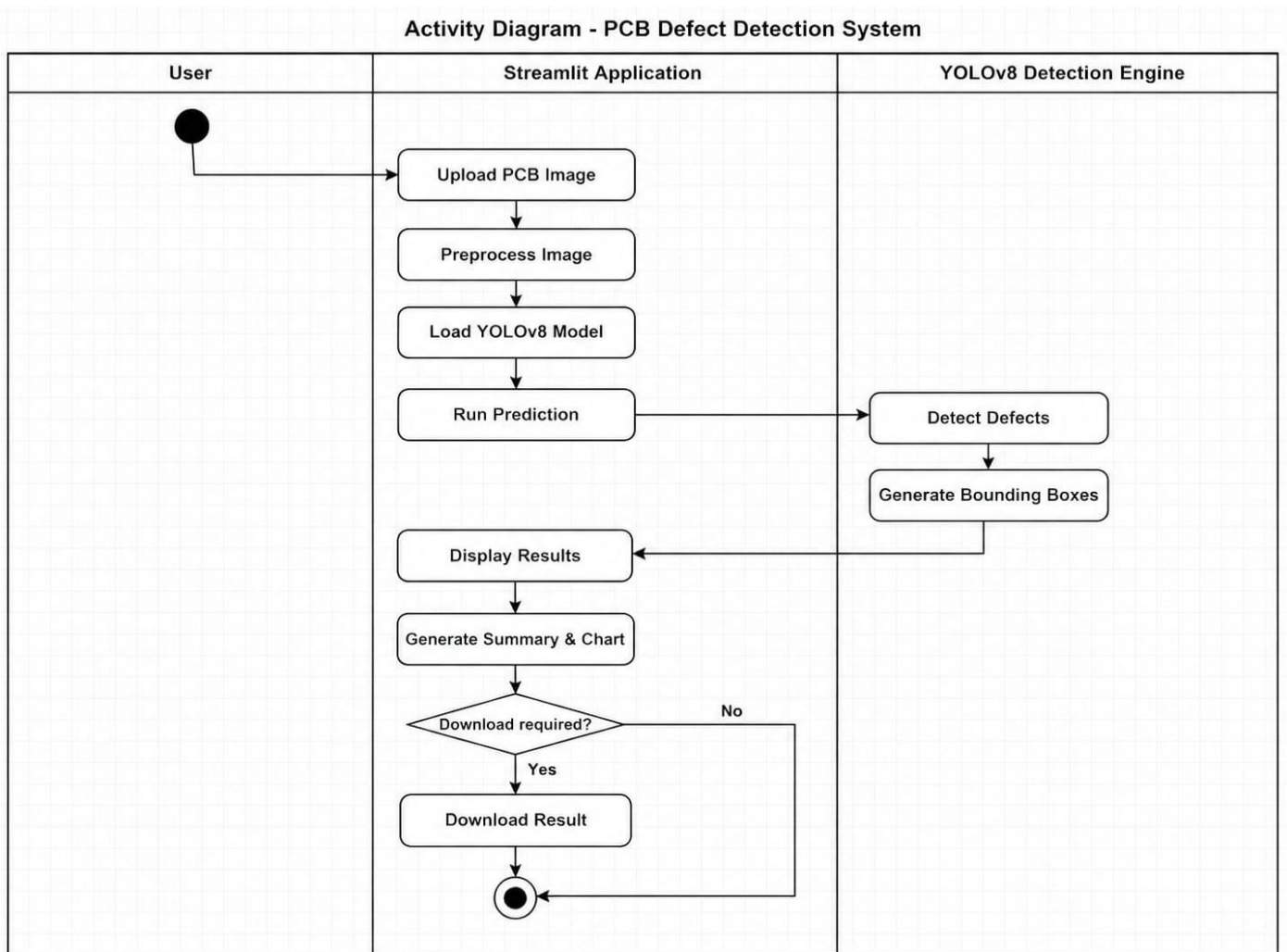


Figure-4.3.5.1 Activity Diagram

4.4 TECHNOLOGY DESCRIPTION

Python

Python is the primary programming language used for developing the proposed PCB Defect Detection System. It provides a simple syntax and extensive support for machine learning, computer vision, and web application development. Python integrates various libraries such as Ultralytics YOLO, OpenCV, NumPy, Pandas, Pillow, and Streamlit, enabling efficient implementation of the entire system from model training to deployment.

DeepPCB Dataset

The DeepPCB dataset is a publicly available benchmark dataset used for training and evaluating the proposed defect detection model. It consists of paired PCB images containing defect-free template images and defective test images. The dataset includes annotations for six common PCB defect classes: Open, Short, Mousebite, Spur, Copper, and Pin-hole. The high-quality annotations enable the model to learn defect patterns accurately and improve detection performance.

YOLOv5

YOLOv5 is one of the object detection models used in this project for comparative analysis. The model was trained using the DeepPCB dataset and evaluated using Precision, Recall, mAP@50, and mAP@50–95. It provided good detection accuracy while serving as a baseline for comparison with the other YOLO models.

YOLOv7

YOLOv7 is an improved object detection model designed to provide better detection accuracy and faster inference than earlier YOLO versions. It was trained using the same DeepPCB dataset and evaluated using identical performance metrics. The model achieved better results than YOLOv5, demonstrating improved localization and classification of PCB defects.

YOLOv8 (Version 8)

YOLOv8 is the final object detection model selected for the proposed PCB Defect Detection System. After comparing the performance of YOLOv5, YOLOv7, and YOLOv8, YOLOv8 achieved the highest Precision, Recall, mAP@50, and mAP@50–95 scores. Therefore, it was integrated into the Streamlit application for real-time detection of PCB defects. The model accurately identifies six different defect types and displays the detected defects with bounding boxes and confidence scores.

Streamlit

Streamlit is an open-source Python framework used to develop the graphical user interface of the PCB Defect Detection System. It provides a simple and interactive web application through which users can upload PCB images for inspection. After processing, the application displays the original image, detected image with bounding boxes, total defect count, defect summary, defect distribution chart, and allows users to download the final detection result.

OpenCV

OpenCV (Open-Source Computer Vision Library) is used for image processing operations within the proposed system. It performs image reading, preprocessing, resizing, format conversion, and visualization of detection results. OpenCV also assists in displaying bounding boxes and labels around detected PCB defects, improving the readability of the output.

Pillow (PIL)

Pillow (Python Imaging Library) is used for loading, manipulating, and converting PCB images before and after prediction. It simplifies image input and output operations and works seamlessly with Streamlit and the YOLOv8 model during inference and result generation.

NumPy

NumPy is used for numerical computations and array manipulation throughout the project. Since images are represented as multidimensional arrays, NumPy enables efficient preprocessing and conversion of prediction outputs into image formats suitable for visualization and downloading.

Pandas

Pandas is used to organize defect information into structured tables and prepare data for visualization. It helps generate the defect summary and supports the creation of bar charts that display the distribution of different PCB defect classes detected in the uploaded image.

PyTorch

PyTorch is the deep learning framework used by the Ultralytics YOLO models. It provides efficient tools for training, validating, and performing inference on deep learning models. PyTorch also supports GPU acceleration, enabling faster training and real-time defect detection using the DeepPCB dataset.

CHAPTER 5

5.1 IMPLEMENTATION

The implementation of the proposed PCB Defect Detection System combines deep learning, computer vision, and web application technologies to perform automatic defect detection on printed circuit boards. The system is developed using Python and integrates the DeepPCB dataset, YOLO object detection models, OpenCV, and Streamlit. Initially, the DeepPCB dataset is preprocessed and used to train three object detection models, namely YOLOv5, YOLOv7, and YOLOv8. After comparing their performance using Precision, Recall, mAP@50, and mAP@50–95, YOLOv8 is selected as the final model. The trained model is then deployed through a Streamlit-based web application for real-time PCB defect detection.

Dataset Preparation

The DeepPCB dataset is used for training and evaluation. It consists of paired PCB images containing template images and defective images with annotations for six defect classes: Open, Short, Mousebite, Spur, Copper, and Pin-hole. The images are preprocessed, resized, and converted into YOLO annotation format. The dataset is divided into training and validation sets before model training.

Model Training

Three object detection models—YOLOv5, YOLOv7, and YOLOv8—are trained using the prepared DeepPCB dataset. All models are trained using the same training and validation data to ensure a fair comparison. Their performance is evaluated using Precision, Recall, mAP@50, and mAP@50–95. Based on the experimental results, YOLOv8 achieves the highest overall detection accuracy and is selected as the final model for deployment.

PCB Defect Detection Using YOLOv8

The trained YOLOv8 model is loaded into the application using the best.pt weight file. When a user uploads a PCB image, the model analyzes the image and detects defects by drawing bounding boxes around the defective regions. The system identifies six defect classes: Open, Short, Mousebite, Spur, Copper, and Pin-hole. The detected image is then displayed along with the total number of defects and a summary of detected defect types.

Image Processing

OpenCV and Pillow (PIL) are used to perform image preprocessing and visualization. The uploaded PCB image is converted into a suitable format before prediction. After inference, bounding boxes and class labels generated by the YOLOv8 model are displayed on the output image. The processed image is also prepared for downloading by the user.

Performance Evaluation

The trained models are evaluated using standard object detection metrics including Precision, Recall, mAP@50, and mAP@50–95. These metrics measure the ability of each model to accurately detect PCB defects. Experimental comparison shows that YOLOv8 provides superior detection performance compared to YOLOv5 and YOLOv7, making it the final model selected for deployment.

User Interface Design

The graphical user interface is developed using Streamlit, providing an interactive platform for PCB defect inspection. The application allows users to:

- Upload PCB images.
- Perform real-time defect detection.
- View the original and detected images.
- Display the total number of detected defects.

- Display the defect summary.
- View the defect distribution chart.
- Download the processed detection result.
- Prediction Output

Once the uploaded image is processed by the trained YOLOv8 model, the application displays the detected defects with bounding boxes and confidence scores. If defects are present, the system generates a defect summary and a bar chart showing the distribution of defect classes. If no defects are detected, the application indicates that the PCB is healthy.

Deployment

The final trained model (best.pt) is integrated into the Streamlit application. The application performs inference in real time, allowing users to inspect PCB images quickly and accurately without requiring specialized hardware or software. The lightweight deployment makes the system suitable for educational demonstrations and industrial inspection tasks.

5.2 MODULES:

Dataset Preparation Module

The Dataset Preparation Module is responsible for organizing the DeepPCB dataset before training. It includes collecting PCB images, verifying annotations, converting annotation files into YOLO format, and dividing the dataset into training and validation sets. Proper dataset preparation ensures that the object detection models receive high-quality labeled data for accurate learning.

Image Upload Module

The Image Upload Module provides an interface through which users can upload PCB images for inspection. The Streamlit application accepts image formats such as JPG, JPEG, and PNG. The uploaded image is temporarily stored and prepared for defect detection using the trained YOLOv8 model.

Image Preprocessing Module

The Image Preprocessing Module prepares the uploaded PCB image before prediction. The image is resized, converted into the required format, and normalized if necessary. This preprocessing ensures compatibility with the YOLO model and improves detection performance.

Model Training Module

The Model Training Module trains three object detection models—YOLOv5, YOLOv7, and YOLOv8—using the DeepPCB dataset. Each model learns to identify six PCB defect classes: Open, Short, Mousebite, Spur, Copper, and Pin-hole. The trained models are evaluated using Precision, Recall, mAP@50, and mAP@50–95. Based on the evaluation results, YOLOv8 achieves the highest performance and is selected for deployment.

PCB Defect Detection Module

The PCB Defect Detection Module performs real-time defect detection using the trained YOLOv8 model. The uploaded PCB image is analyzed, defects are localized using bounding boxes, and each detected defect is classified into one of the six predefined categories. This module forms the core functionality of the proposed system.

Result Visualization Module

The Result Visualization Module presents the detection results to the user in an understandable format. It displays the original PCB image, the detected image with bounding boxes, the total number of detected defects, a defect summary, and a bar chart representing the distribution of different defect classes.

Download Module

The Download Module enables users to download the processed PCB image after defect detection. The detected image containing annotated bounding boxes is converted into PNG format and made available through the Streamlit interface for easy downloading and future reference.

User Interface Module

The User Interface Module is developed using Streamlit to provide a simple and interactive environment for PCB inspection. Users can upload PCB images, perform real-time defect detection, view detection results, analyze defect statistics, and download the processed image through a single web interface.

5.3 EXECUTABLE CODE

convert.py

```
import os

# Dataset path
dataset_path = r"dataset\DeepPCB-master\DeepPCB-master\PCBData"

# Output YOLO labels
output_folder = r"dataset\labels\train"

os.makedirs(output_folder, exist_ok=True)

# Image size
img_width = 640
img_height = 640

# Loop through all group folders
for group in os.listdir(dataset_path):

    group_path = os.path.join(dataset_path, group)

    if not os.path.isdir(group_path):
        continue

    annotation_folder = os.path.join(group_path, group.replace("group", "") + "_not")

    if not os.path.exists(annotation_folder):
        continue

    for file_name in os.listdir(annotation_folder):

        file_path = os.path.join(annotation_folder, file_name)

        with open(file_path, "r") as f:
            lines = f.readlines()

        yolo_lines = []

        for line in lines:

            x1, y1, x2, y2, cls = map(int, line.split())

            # Convert to YOLO format
            x_center = ((x1 + x2) / 2) / img_width
            y_center = ((y1 + y2) / 2) / img_height
```

```

width = (x2 - x1) / img_width
height = (y2 - y1) / img_height

# YOLO line
yolo_line = f'{cls-1} {x_center} {y_center} {width} {height}'

yolo_lines.append(yolo_line)

# Save YOLO label
output_file = os.path.join(output_folder, file_name + ".txt")

with open(output_file, "w") as f:
    f.write("\n".join(yolo_lines))

print("Conversion Completed")

```

train.py

```

from ultralytics import YOLO

# Load pre-trained YOLOv8 model
model = YOLO("yolov8n.pt")

# Train the model
results = model.train(
    data="/kaggle/working/kaggle_data.yaml",
    epochs=50,
    imgsz=640,
    batch=16
)

# Load the best trained model
model = YOLO("/kaggle/working/runs/detect/train/weights/best.pt")

# Perform prediction on validation images
results = model.predict(
    source="/kaggle/input/datasets/bavyasrisaithatipudi/pcb-
defectdetection/PCB_Detection/full_dataset/images/val",
    save=True,
    conf=0.25
)

```

app.py

```
import asyncio
asyncio.set_event_loop_policy(asyncio.WindowsSelectorEventLoopPolicy())

import streamlit as st
from ultralytics import YOLO
from PIL import Image
import tempfile
import io
import numpy as np
from collections import Counter
import pandas as pd
import time

# Page settings
st.set_page_config(page_title="PCB Defect Detection", layout="wide")

# Custom CSS Styling
st.markdown("""
<style>
.main {
  background-color: #0E1117;
}

div[data-testid="metric-container"] {
  background-color: #1E2530;
  border: 1px solid #2E3B4E;
  padding: 15px;
  border-radius: 15px;
}
</style>
""", unsafe_allow_html=True)

# Hero Header
st.markdown("""
# 🕒 PCB Defect Detection Dashboard
### AI-powered defect inspection using YOLOv8
Upload a PCB image to detect manufacturing defects in real-time.
""")

# Sidebar
st.sidebar.success("Model Loaded Successfully")
st.sidebar.write("Supported Defects:")
st.sidebar.write("• open")
st.sidebar.write("• short")
```

```

st.sidebar.write("• mousebite")
st.sidebar.write("• spur")
st.sidebar.write("• copper")
st.sidebar.write("• pin-hole")

# Load model
model = YOLO("best.pt")

# Upload image
uploaded_file = st.file_uploader(
    "Upload PCB Image",
    type=["jpg", "jpeg", "png"]
)

if uploaded_file is not None:
    image = Image.open(uploaded_file)

    # Loading spinner + processing time
    with st.spinner("Analyzing PCB defects..."):
        with tempfile.NamedTemporaryFile(delete=False, suffix=".jpg") as tmp:
            image.save(tmp.name)

            start = time.time()
            results = model.predict(tmp.name, conf=0.25)
            end = time.time()

        processing_time = round(end - start, 3)

    result = results[0]
    predicted = result.plot()

    # Convert predicted image for download
    predicted_pil = Image.fromarray(np.uint8(predicted))
    buf = io.BytesIO()
    predicted_pil.save(buf, format="PNG")

    # Side-by-side image display
    col1, col2 = st.columns(2)

    with col1:
        st.image(image, caption="Original PCB", width=500)

    with col2:
        st.image(predicted, caption="Detected PCB", width=500)

    total_defects = len(result.boxes)

```

```

st.write("## Detection Results")

# Metric Cards (4 cards now)
m1, m2, m3, m4 = st.columns(4)
m1.metric("Total Defects", total_defects)
m2.metric("Model", "YOLOv8")
m3.metric("mAP@50", "98.8%")
m4.metric("Inference Time", f"{processing_time}s")

# Severity Classification
if total_defects == 0:
    st.success("PCB is Healthy 🟢")
elif total_defects <= 3:
    st.warning("Minor Defects Detected ⚠️")
elif total_defects <= 6:
    st.warning("Moderate Damage 🟡")
else:
    st.error("Critical PCB Damage 🚨")

if total_defects > 0:
    # Count defect types
    class_names = result.names
    detected_classes = [class_names[int(box.cls)] for box in result.bboxes]
    defect_counts = Counter(detected_classes)

    st.write("### Defect Summary")
    for defect, count in defect_counts.items():
        st.write(f"• {defect}: {count}")

    # Bar Chart
    chart_data = pd.DataFrame({
        "Defect": list(defect_counts.keys()),
        "Count": list(defect_counts.values())
    })

    st.write("### Defect Distribution")
    st.bar_chart(chart_data.set_index("Defect"))

# Download button
st.download_button(
    label="📄 Download Detection Result",
    data=buf.getvalue(),
    file_name="pcb_detection_result.png",
    mime="image/png"
)

```

data.yaml

path: /kaggle/input/datasets/bavyasrisaithatipudi/pcb-defectdetection/PCB_Detection/full_dataset

train: images/train

val: images/val

nc: 6

names:

- 0: open
- 1: short
- 2: mousebite
- 3: spur
- 4: copper
- 5: pin-hole

requirements.txt

- streamlit
- ultralytics
- torch
- opencv-python
- numpy
- pandas
- pillow

CHAPTER -6

RESULTS

Model Performance Comparison

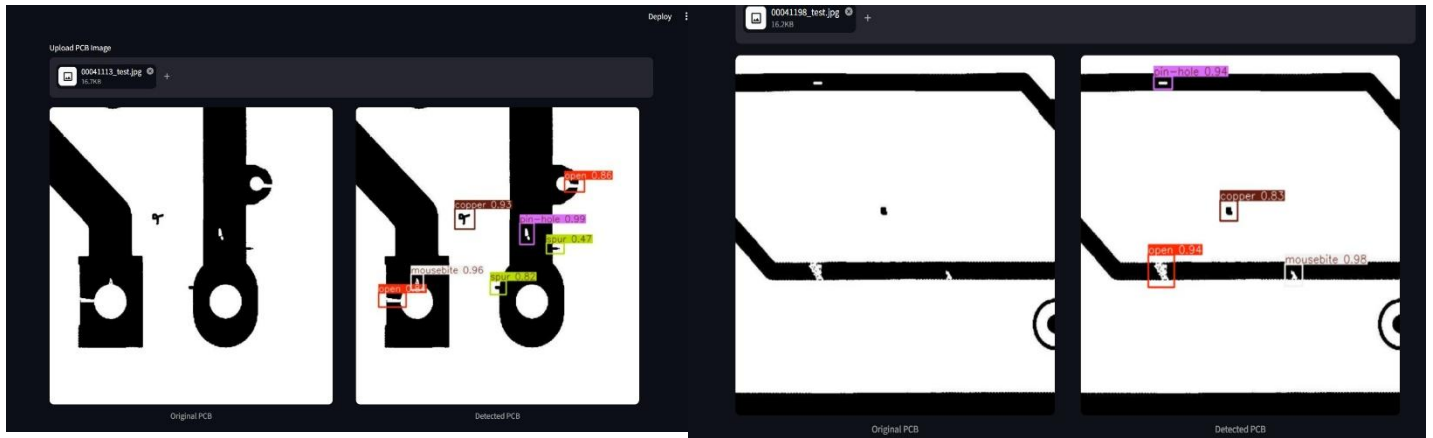
The performance of YOLOv5, YOLOv7, and YOLOv8 was evaluated on the DeepPCB dataset. The models were compared using Precision, Recall, and mAP metrics. Based on the evaluation, YOLOv8 achieved the best overall performance and was selected for deployment.

Model	Precision	Recall	mAP@50-95	mAP@50-95	Training Time
YOLOv5	97.7%	95,2%	98.2%	69.0%	23.2 min
YOLOv7	97.6%	95.8%	98.4%	73%	18 min
YOLOv8	97.1%	96.3%	98.8%	76.8%	11.8 min

Detection Results

The selected YOLOv8 model was deployed using a Streamlit application. The following screenshots show the detection results obtained for different PCB images.



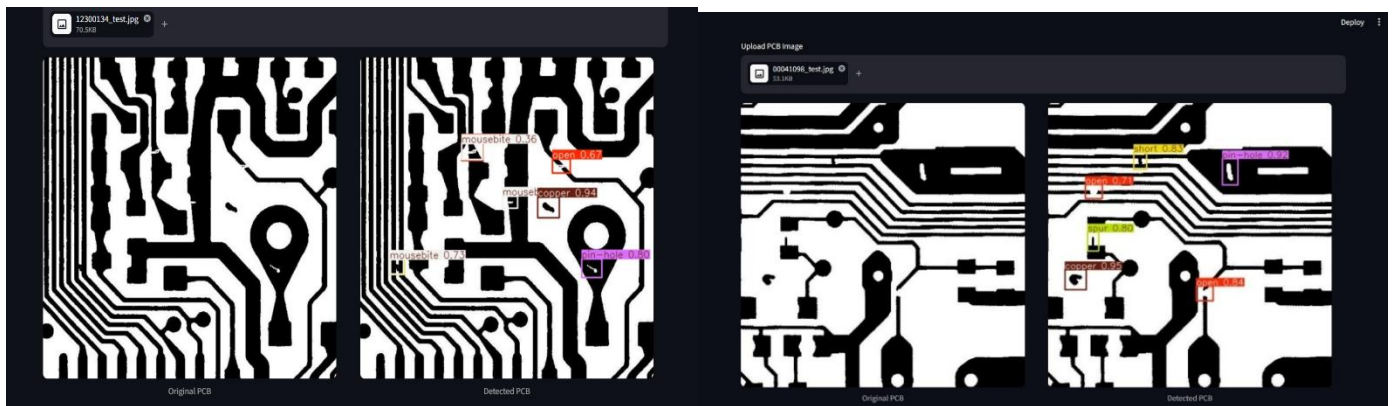


Input

Output

Input

Output

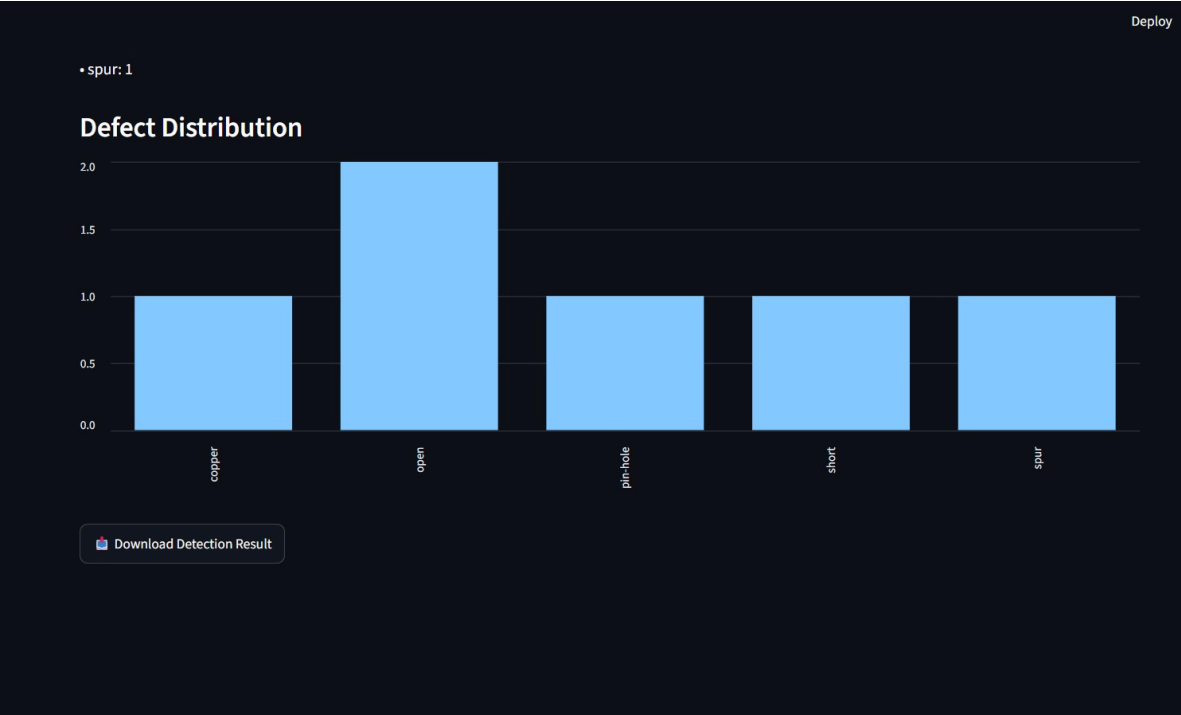
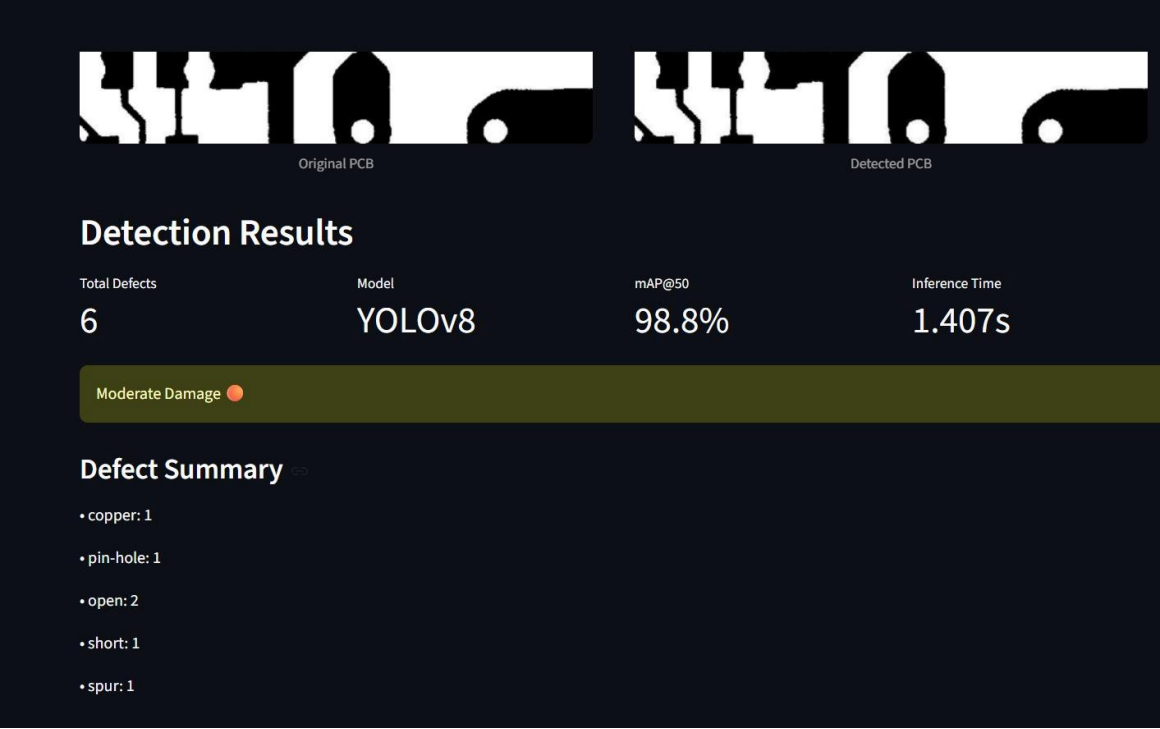


Input

Output

Input

Output



CHAPTER 7

7.1 CONCLUSION

This project presents an intelligent PCB defect detection system based on deep learning using the YOLOv8 object detection model. The DeepPCB dataset was used to train and evaluate multiple YOLO architectures, including YOLOv5, YOLOv7, and YOLOv8. After comparing their performance using Precision, Recall, and mAP metrics, YOLOv8 was selected as the final model due to its superior detection accuracy and inference speed.

The developed Streamlit application enables users to upload PCB images and automatically detect manufacturing defects such as open circuits, shorts, mousebites, spurs, copper defects, and pin-holes. The system displays annotated output images, defect statistics, and downloadable detection results through an interactive interface. The proposed solution demonstrates that deep learning-based inspection can significantly improve the speed, accuracy, and reliability of PCB quality assessment while reducing manual inspection effort.

7.2 FUTURE SCOPE

- **Real-Time Video Inspection:** Extend the system to detect PCB defects from live camera feeds for automated inspection on production lines.
- **Support for More Defect Types:** Train the model with larger and more diverse datasets to identify additional PCB manufacturing defects with higher accuracy.
- **Deployment on Embedded Devices:** Deploy the trained YOLOv8 model on edge devices such as NVIDIA Jetson or Raspberry Pi for real-time industrial applications.

- **Cloud-Based Inspection System:** Develop a web-based platform where users can upload PCB images remotely and receive defect analysis and reports.
- **Integration with Industrial Automation:** Integrate the defect detection system with robotic manufacturing systems to automatically separate defective PCBs from production.

REFERENCES

- [1]. Sanli Tang, Fan He, Xiaolin Huang, and Jie Yang, “Online PCB defect detector on a new PCB defect dataset,” arXiv preprint arXiv:1902.06197, 2019.
- [2]. Weibo Huang and Peng Wei, “A PCB dataset for defects detection and classification,” Journal of Latex Class Files, vol. 14, no. 8, pp. 1–9, Aug. 2018.
- [3]. JiaYou Lim, JunYi Lim, Vishnu Monn Baskaran, and Xin Wang, “A deep context learning based PCB defect detection model with anomalous trend alarming system,” Results in Engineering, vol. 17, p. 100968, 2023.
- [4]. Jiaxin Liu, Bingyu Kang, Chao Liu, Xunhui Peng, and Yan Bai, “YOLO-BFRV: An efficient model for detecting printed circuit board defects,” Sensors, vol. 24, no. 18, p. 6055, 2024.

PLAGIARISM TEST

Paraphrasing Tool

Plagiarism Checker

Reverse Image Search

EN Login

Free Tools

Pricing

Plagiarism Checker

Duplichecker detects plagiarism from your text more accurately

Accurate Reports

Deep search

Support

Upto 25,000 words

No Ads

Try Premium

The Image Upload Module provides an interface through which users can upload PCB images for inspection. The Streamlit application accepts image formats such as JPG, JPEG, and PNG. The uploaded image is temporarily stored and prepared for defect detection using the trained YOLOv8 model.

Image Preprocessing Module

The Image Preprocessing Module prepares the uploaded PCB image before prediction. The image is resized, converted into the required format, and normalized if necessary. This preprocessing ensures compatibility with the YOLO model and improves detection performance.

Model Training Module

The Model Training Module trains three object detection models—YOLOv5, YOLOv7, and YOLOv8—using the DeepPCB dataset. Each model learns to identify six PCB defect classes: Open, Short, Mousebite, Spur, Copper, and Pin-hole. The trained models are evaluated using Precision, Recall, mAP@50, and mAP@50-95. Based on the evaluation results, YOLOv8 achieves the highest performance and is selected for deployment.

24266 / 10,000 chars - 3520 Words

Scan Settings

General Rules

Exclude Quotes PRO

Exclude Bibliography PRO

AI & Integrity

AI Detection

Fingerprint PRO

Scan Mode

Normal Scan

Deep Scan PRO

Academic Scan PRO

Check Plagiarism via URL

Exclude URLs

Check Plagiarism

Check Grammar

4%

Plagiarized

Exact Match

4%

Partial Match

0%

96%

Unique

Original Content

35